

# OS Unit IV - Memory Management

---

## Unit IV - Memory Management

---

Operating System | MSBTE K-Scheme | Diploma CO Branch | TY

---

### 4.1 Basic Memory Management

---

Memory management is the function of the OS that handles or manages primary memory. It keeps track of each memory location, whether it is allocated to a process or is free, and decides how memory is allocated among competing processes.

**Goals of Memory Management:** - Efficient utilization of memory - Keep track of which parts of memory are in use and by whom - Allocate and deallocate memory space as needed - Protect processes from interfering with each other's memory

---

#### Partitioning - Fixed and Variable

Memory partitioning divides main memory into sections to hold processes.

**1. Fixed (Static) Partitioning** - Main memory is divided into a **fixed number of partitions** of fixed size at system startup (or boot time). These sizes do not change during system operation. - Each partition may hold exactly one process. - The degree of multiprogramming is limited by the number of partitions.

**Advantages:** - Simple to implement; low OS overhead.

**Disadvantages:** - **Internal Fragmentation** - if a process is smaller than the partition, the leftover space inside the partition is wasted. - A process larger than the largest partition cannot be run. - Limits the number of active processes to the number of partitions.

**2. Variable (Dynamic) Partitioning** - Partitions are created **dynamically** at run time, exactly matching the size of each process's memory request. - The size and number of partitions may vary over time as processes enter and leave.

**Advantages:** - No internal fragmentation, since each partition is exactly the size needed. - Better memory utilization than fixed partitioning.

**Disadvantages:** - **External Fragmentation** - as processes are loaded and removed, small free blocks scattered throughout memory may become too small individually to satisfy new requests, even though total free memory is sufficient. - Requires more complex memory management (tracking variable-sized free blocks).

**Comparison Table:** | Fixed Partitioning | Variable Partitioning | |—|—| | Partition sizes fixed at boot | Partition sizes set at run time, based on process size | | Internal fragmentation | External fragmentation | | Simple to manage | More complex to manage | | Limited degree of multiprogramming | Flexible degree of multiprogramming |

---

#### Free Space Management Techniques

The OS must keep track of free (unallocated) memory blocks so it can allocate them to new processes. Two common techniques:

**1. Bit Map (Bit Vector)** - Memory is divided into fixed-size allocation units (e.g., blocks/clusters). - Each unit is represented by **one bit**: **0** = free, **1** = allocated (or vice versa). - A bit map (bit vector/bitmap) is a string of these bits, one per memory unit.

**Example:** If memory blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, 27 are free, the bit map would show 0's for these positions and 1's for allocated positions.

**Advantages:** - Simple technique, relatively easy to implement. - Finding the first free block or a set of n consecutive free blocks is straightforward by scanning the bitmap.

**Disadvantages:** - Searching the bitmap for a free block can be slow (linear scan) unless hardware instructions help. - Bitmap itself takes up memory, though relatively small.

**2. Linked List** - Free memory blocks are linked together using a **linked list**, where each node stores the starting address and size (length) of a free block, plus a pointer to the next free block. - Allocated blocks are simply skipped/removed from this list.

**Advantages:** - No wasted memory for a large bitmap; can represent variable-sized holes directly.

**Disadvantages:** - Traversing the list to find a suitably-sized free block takes more time (especially for algorithms like Best Fit which must scan the entire list). - More overhead in maintaining pointers.

**Comparison Table:** | Bit Map | Linked List | |—|—| | Uses 1 bit per block | Uses a node (address + size + pointer) per free block | | Fast to check if a block is free | Must traverse list to find a block | | Fixed overhead regardless of fragmentation | Overhead grows with fragmentation (more free blocks = more nodes) |

---

## 4.2 Swapping, Compaction, Fragmentation, Partitioning Algorithms

---

### Swapping

- **Swapping** is a technique in which a process is temporarily “swapped out” from main memory to a backing store (disk), and later “swapped in” back to memory to continue execution.
- Used by the **medium-term scheduler** to increase the degree of multiprogramming.
- Allows the total memory requirements of all processes to exceed the actual physical memory available.

**Process:** 1. The OS selects a process to swap out (e.g., a blocked or low-priority process). 2. Its entire memory image is copied to a swap space on disk. 3. Later, it is swapped back in, often to the same or a different memory location.

**Drawback:** Swapping involves significant I/O overhead (transferring entire process images to/from disk), which can be time-consuming.

---

### Fragmentation

Fragmentation refers to wasted memory that cannot be effectively used. There are two types:

1. **Internal Fragmentation** – occurs when memory is allocated in fixed-size blocks/partitions, and the allocated block is slightly larger than the requested memory, wasting the leftover space **within** the partition.
2. **External Fragmentation** – occurs when free memory is broken into small, non-contiguous blocks scattered across memory, so that even though the total free memory is enough, no single block is large enough to satisfy a request.

---

### Compaction

- **Compaction** is a technique to solve external fragmentation by **shuffling memory contents** to place all free memory together as one large contiguous block.
- This is done by moving all allocated (occupied) partitions towards one end of memory, and all free memory towards the other end.

**Requirements:** - Relocation must be **dynamic** (using base/relocation registers), since compaction changes the physical addresses of processes at run time.

**Drawback:** Compaction is expensive in terms of CPU time, as it involves copying large blocks of memory

content.

---

## Partitioning (Dynamic Storage Allocation) Algorithms

When using variable/dynamic partitioning, when a process requests memory of size  $n$ , the OS must select a free hole (block) from the set of available holes. Three common strategies:

- 1. First Fit** - Allocate the **first hole** found that is big enough to satisfy the request, scanning from the beginning of memory (or from where the previous search left off). - **Fast** — stops searching as soon as a suitable hole is found.
- 2. Best Fit** - Allocate the **smallest hole** that is big enough to satisfy the request among all available holes.  
- Requires searching the **entire list** of holes (unless it is sorted by size), which is slower. - Produces the **smallest leftover hole**, which can lead to many tiny, unusable fragments over time.
- 3. Worst Fit** - Allocate the **largest hole** available. - Also requires searching the entire list (unless sorted). - Leaves the **largest leftover hole**, which may be more useful for future requests than the tiny leftovers of Best Fit.

**Comparison Table:** | Algorithm | Search Strategy | Speed | Memory Utilization | |—|—|—|—| | First Fit | First hole big enough | Fastest | Generally good, simple | | Best Fit | Smallest hole big enough | Slowest (full scan) | Best memory usage, but creates tiny unusable fragments | | Worst Fit | Largest hole available | Slow (full scan) | Reduces tiny fragments, but wastes large holes quickly |

**Note:** Simulation studies generally show that both **First Fit** and **Best Fit** are better than Worst Fit in terms of both time and storage utilization; First Fit is usually faster.

---

## 4.3 Non-Contiguous Memory Management Techniques

---

Contiguous allocation requires each process to occupy a single continuous block of memory, which leads to fragmentation. **Non-contiguous allocation** allows a process's memory to be divided and scattered across different areas of physical memory.

### Paging

- **Paging** is a memory management scheme that eliminates the need for contiguous allocation of physical memory, thereby avoiding external fragmentation.
- **Physical memory** is divided into fixed-size blocks called **frames**.
- **Logical (virtual) memory** is divided into blocks of the **same fixed size**, called **pages**.
- When a process is to be executed, its pages are loaded into any available frames (not necessarily contiguous).

**Address Translation:** - A logical address generated by the CPU is divided into two parts: **Page Number (p)** and **Page Offset (d)**. - The **Page Number** is used as an index into a **Page Table**, which contains the **base address (frame number)** of each page in physical memory. - The **Page Table** combined with the offset gives the final **physical address**.

**Diagram (described):**

```
Logical Address = [Page Number | Offset]
Page Table: Page Number --> Frame Number
Physical Address = [Frame Number | Offset]
```

**Advantages:** - No external fragmentation (any free frame can be used for any page). - Simplifies memory allocation.

**Disadvantages:** - **Internal fragmentation** may occur in the last page of a process (if the process size is not an exact multiple of the page size). - Requires extra memory access to look up the page table (solved using a **Translation Lookaside Buffer, TLB**, a fast hardware cache for page table entries).

---

## Segmentation

- **Segmentation** is a memory management scheme that supports the user's/programmer's view of memory — a program is divided into **variable-sized logical segments**, such as main program, functions, stack, and data, based on logical grouping rather than fixed size.
- Each segment has a **name** (or number) and a **length**.

**Address Translation:** - A logical address consists of two parts: **Segment Number (s)** and **Offset (d)**. - A **Segment Table** maps each segment number to a **base address** (starting physical address) and a **limit** (length of the segment). - The physical address = segment's base address + offset (checked against the limit to prevent out-of-bounds access).

**Diagram (described):**

```
Logical Address = [Segment Number | Offset]
Segment Table: Segment Number --> (Base, Limit)
Physical Address = Base + Offset (if Offset < Limit)
```

**Advantages:** - Matches the logical view of a program (code, data, stack as separate units), which makes protection and sharing easier (e.g., a shared code segment). - Segments can grow independently.

**Disadvantages:** - Since segments are of **variable size**, segmentation suffers from **external fragmentation**, similar to variable partitioning.

**Paging vs Segmentation:** | Paging | Segmentation | |—|—| | Divides memory into fixed-size pages/frames | Divides memory into variable-sized logical segments | | No external fragmentation | External fragmentation possible | | May cause internal fragmentation | No internal fragmentation | | Invisible to the programmer (system-oriented) | Reflects logical program structure (user-oriented) |

---

## 4.4 Virtual Memory

### Basics

- **Virtual Memory** is a technique that allows the execution of processes that may not be **completely** in main memory — it separates a user's logical memory from physical memory.
- This lets programs be much larger than the actual physical memory available, because only the portions of a program currently needed are kept in memory.
- Virtual memory is typically implemented using **demand paging** (or demand segmentation).

**Benefits:** - Programs can be larger than physical memory. - More programs can run concurrently, increasing CPU utilization and throughput. - Less I/O is needed to load or swap programs into memory, so each program runs faster.

---

### Demand Paging

- A **demand-paging** system is similar to a paging system with swapping, where processes reside in secondary memory (disk).
- Instead of loading an entire process into memory before execution, pages are loaded **only when they are needed (demanded)** during execution — this is called **lazy swapper** behavior.
- Each page table entry has a **valid-invalid bit** (or present/absent bit) to indicate whether the page is currently in memory (valid) or not (invalid — meaning either the page is on disk or is not part of the process's logical address space).

**Working:** 1. The CPU tries to access a page. 2. The Memory Management Unit (MMU) checks the valid-invalid bit in the page table. 3. If the page is in memory (valid), address translation proceeds normally. 4. If the page is **not** in memory (invalid), a **page fault** occurs.

---

### Page Fault

A **page fault** occurs when a process tries to access a page that is **not currently loaded** in main memory.

**Steps to handle a page fault:** 1. The OS checks an internal table (kept with the PCB) to determine whether the reference was valid or invalid. 2. If invalid (the process tried to access memory outside its address space), the process is terminated. 3. If valid but the page is not yet in memory, the OS proceeds to bring it in: - Find a **free frame** in physical memory (using a free-frame list). - **Schedule a disk operation** to read the required page into the newly allocated frame. - When the disk read completes, **update the page table** to mark the page as valid and record the frame number. - **Restart the instruction** that caused the page fault, so the process can continue.

**If no free frame is available:** the OS must select a page currently in memory to remove (using a **page replacement algorithm**) to make room for the new page.

**Effective Access Time (EAT)** is affected by the page fault rate — since servicing a page fault (disk I/O) is far slower than a normal memory access, a high page fault rate can drastically slow down a system, an effect known as **thrashing** (excessive paging activity that severely degrades performance).

---

## 4.5 Page Replacement Algorithms

---

When a page fault occurs and no free frame is available, the OS must choose an existing page to **replace (evict)** from memory. This decision is made by a **page replacement algorithm**. The goal is to minimize the **page fault rate**.

### First In First Out (FIFO)

- Replaces the **oldest page** in memory — the page that has been in memory the **longest** — regardless of how recently or frequently it was used.
- Implemented using a **queue**: new pages are added to the tail; when replacement is needed, the page at the head (oldest) is removed.

**Advantages:** Simple to understand and implement.

**Disadvantages:** - Performance is not always good — an old page might still be in frequent use. - Suffers from **Belady's Anomaly**: in some cases, increasing the number of frames can actually **increase** the number of page faults (unusual and counter-intuitive behavior specific to FIFO).

---

### Least Recently Used (LRU)

- Replaces the page that has **not been used for the longest period of time** — i.e., the page that was least recently accessed.
- Based on the idea that pages used recently are likely to be used again soon (principle of locality), so the page unused for the longest time is a good candidate for replacement.

**Implementation approaches:** 1. **Counters** – Each page table entry has a time-of-use field; the CPU has a logical clock/counter incremented on every memory reference. On each page access, the counter value is copied to the page table entry for that page. The page with the smallest counter (oldest use) is replaced. 2. **Stack** – Maintain a stack of page numbers. Whenever a page is referenced, it is moved to the top of the stack. The page at the bottom of the stack is the least recently used.

**Advantages:** Does **not** suffer from Belady's Anomaly; generally performs well since it approximates the optimal algorithm using past behavior.

**Disadvantages:** Requires special hardware support (counters or stack updates on every memory reference) for efficient implementation; can be costly without hardware assistance.

---

### Optimal Page Replacement

- Replaces the page that **will not be used for the longest period of time in the future**.
- Guarantees the **lowest possible page fault rate** for a fixed number of frames — it is the theoretical

best algorithm.

**Limitation:** Requires **future knowledge** of the reference string (which pages will be requested and when), which is generally **impossible** to know in advance in a real operating system.

**Use:** Mainly used as a **benchmark/comparison standard** to measure how close other practical algorithms (like LRU or FIFO) come to the best possible performance, often tested via simulation with a known reference string.

**Example (reference string comparison):** Given reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 with 3 frames: - **FIFO** results in more page faults due to blindly removing the oldest page. - **LRU** performs better by considering recent usage. - **Optimal** achieves the fewest page faults by “looking ahead,” but is not implementable in practice.

**Comparison Table:** | Algorithm | Basis of Replacement | Belady's Anomaly | Practical? | |—|—|—| | FIFO | Oldest page in memory | Yes | Yes, but weak performance | | LRU | Least recently used page | No | Yes, needs hardware support | | Optimal | Page not needed for longest future time | No | No (needs future knowledge) — used only as a benchmark |

---

## Quick Revision Summary

---

- **Fixed partitioning** → internal fragmentation; **Variable partitioning** → external fragmentation.
- **Free space management:** Bit Map (1 bit/block) and Linked List (nodes for free blocks).
- **Swapping** moves processes between memory and disk; **Compaction** solves external fragmentation by shifting memory content; **Fragmentation** = wasted memory (internal/external).
- **Partitioning algorithms:** First Fit (fast), Best Fit (smallest leftover, slow), Worst Fit (largest leftover, slow).
- **Paging:** fixed-size pages/frames, no external fragmentation, uses Page Table. **Segmentation:** variable-size logical segments, matches program structure, uses Segment Table (Base, Limit).
- **Virtual Memory:** allows processes larger than physical memory; implemented via **Demand Paging** — pages loaded only when needed.
- **Page Fault:** occurs when a required page is not in memory; handled by locating/loading the page from disk and updating the page table.
- **Page Replacement Algorithms:** FIFO (oldest page, suffers Belady's Anomaly), LRU (least recently used, no anomaly, needs hardware support), Optimal (best possible, needs future knowledge — used only as a benchmark).